UNLOX ACADEMY

VLSI Design Program, September 2026 Batch

MINOR PROJECT REPORT

PROJECT 01

TRAFFIC LIGHT CONTROLLER

(Timer-Based and Sensor-Based Designs)

Modelled in Verilog HDL using Finite State Machines

Student Name	Thipperudra K T
Roll No.	4SM23EE037
Date	19 October 2026
Instructor	Kuldeep Sompura

Declaration

I, **Thipperudra K T**, a participant of the Unlox Academy VLSI Design program (September 2026 batch), hereby declare that this report titled **"Traffic Light Controller (Timer and Sensor) using Verilog HDL"** has been prepared by me as part of the Minor Project (Project 01) of the program. The designs, state diagrams, source code and simulation waveforms discussed here are those provided in the program assignment, and they have been studied, explained and documented in this report in my own words.

The report has not been submitted to any other institution or program for any award or evaluation.

Date: 19 October 2026

Thipperudra K T

Acknowledgement

I sincerely thank **Unlox Academy** for the VLSI Design program and for giving me the opportunity to work on a practical digital design project. The structured sessions on digital logic, Verilog HDL and simulation made it possible to understand how a real-world control problem such as a traffic signal can be described as hardware.

I am grateful to the mentors and trainers of the program for their clear explanations of finite state machines, counters and test-bench writing, and for the assignment material that formed the basis of this project. I also thank my fellow participants for the discussions that helped me clarify several doubts, and my family for their constant support and encouragement.

Thipperudra K T

Abstract

Traffic signals are a classic example of a sequential control system: the output depends not only on the present inputs but also on the history of what has already happened at the junction. Such systems are naturally described as **Finite State Machines (FSMs)**. This project presents the design and simulation of a two-road traffic light controller in **Verilog Hardware Description Language (HDL)** using two different control strategies.

The first design, *Traffic_Light_Time*, is a **timer-based controller**. It uses a six-state FSM (S0 to S5) and a 4-bit counter. Long states (S0 and S3) last 16 clock cycles and the short transition states (S1, S2, S4 and S5) last 4 clock cycles each, giving a complete cycle of 48 clock cycles. A six-bit output vector *lights[5:0]* drives the red, yellow and green lamps of both roads.

The second design, *Traffic_Light_Sensor*, is a **sensor-based controller**. It uses a four-state FSM (S0 to S3) with two sensor inputs, SNS1 and SNS2, which decide whether the controller stays in a state or moves on. It drives two 2-bit outputs, TL1 and TL2, that encode green, yellow and red for each road.

Both designs use a synchronous clock and an asynchronous clear. The test benches, simulation waveforms, state tables, timing calculations and a design comparison are documented in this report. The project demonstrates how state-machine modelling, parameterised state encoding, counter-based timing and sensor-driven transitions can be combined to describe practical control hardware in Verilog.

Keywords: Verilog HDL, Finite State Machine, Traffic Light Controller, Counter, Sensor, Moore Machine, Simulation, Testbench.

Table of Contents

Declaration 2

Acknowledgement 3

Abstract 4

Table of Contents 5

1. Introduction 7

1.1 Background . . . 7

1.2 Motivation . . . 7

1.3 Scope of the Project . . . 7

1.4 Organisation of the Report . . . 7

2. Problem Statement and Objectives 8

2.1 Problem Statement . . . 8

2.2 Objectives . . . 8

2.3 Design Requirements . . . 8

3. Theoretical Background 9

3.1 Sequential Circuits and Finite State Machines . . . 9

3.2 Moore and Mealy Machines . . . 9

3.3 Hardware Description Language and Verilog . . . 9

3.4 Counter-Based Timing . . . 10

3.5 Sensor-Driven Control . . . 10

3.6 Synchronous Design and Reset . . . 10

4. Tools and Design Elements 11

4.1 Tools Used . . . 11

4.2 Signals of the Timer-Based Design . . . 11

4.3 Parameters of the Timer-Based Design . . . 11

4.4 Signals and Parameters of the Sensor-Based Design . . . 11

5. Timer-Based Traffic Light Controller 13

5.1 Design Overview . . . 13

5.2 State Diagram . . . 13

5.3 State Table and Output Encoding . . . 13

5.4 Source Code Explanation . . . 14

5.5 Timing Analysis . . . 15

5.6 Working Steps of the Timer-Based Controller . . . 16

6. Sensor-Based Traffic Light Controller 17

6.1 Design Overview 17

6.2 State Diagram 17

6.3 State Transition Table 17

6.4 Output Table 18

6.5 Source Code Explanation 18

6.6 Working Steps of the Sensor-Based Controller 20

7. Testbench and Simulation Results 21

7.1 Timer-Based Testbench 21

7.2 Timer-Based Simulation Waveform 21

7.3 Sensor-Based Testbench 22

7.4 Sensor-Based Simulation Waveform 24

7.5 Summary of Results 24

8. Comparison of the Two Designs 25

9. Advantages, Limitations, Applications and Future Scope 26

9.1 Advantages 26

9.2 Limitations 26

9.3 Applications 26

9.4 Future Scope 27

10. Conclusion 28

References 28

Appendix A: Complete Timer-Based Source Code 29

Appendix B: Complete Sensor-Based Source Code 31

Appendix C: Glossary 32

1. Introduction

1.1 Background

Road intersections need an orderly way of giving vehicles from different directions the right of way. A traffic light does this by displaying a repeating sequence of **red** (stop), **yellow** (get ready or clear the junction) and **green** (go) signals. Behind every signal is a controller that decides which lamp is on, for how long, and what comes next.

In digital electronics this controller is a *sequential circuit*: it stores its present condition in flip-flops and moves from one condition (state) to the next on every clock edge. Modern digital chips such as FPGAs and ASICs are designed by describing this behaviour in a Hardware Description Language like Verilog, simulating it, and then synthesising it into gates and flip-flops.

1.2 Motivation

A traffic light controller is one of the most instructive beginner-to-intermediate VLSI problems because it brings together several important ideas in a small design: state encoding, state transitions, counters for timing, input-dependent transitions (sensors), output decoding, reset behaviour and test-bench based verification. Working through it builds the habits needed for larger designs.

1.3 Scope of the Project

This project covers two controllers for a junction with two roads:

- **Timer-based controller** that changes states purely according to a count of clock cycles.
- **Sensor-based controller** that changes states according to the status of two vehicle sensors, SNS1 and SNS2.

Both are modelled at the Register Transfer Level (RTL) in Verilog and verified by simulation. The scope of this report is the design, behavioural explanation and simulation of the two controllers; physical implementation on an FPGA board or a road-side installation is outside the scope of the project.

1.4 Organisation of the Report

Chapter 2 states the problem and objectives. Chapter 3 gives the theoretical background. Chapter 4 lists the tools and signals used. Chapters 5 and 6 describe the timer-based and sensor-based designs in detail. Chapter 7 presents the testbenches and simulation results, and Chapter 8 compares the two designs. Chapter 9 discusses advantages, limitations, applications and future scope, followed by the conclusion, references and appendices containing the source code.

2. Problem Statement and Objectives

2.1 Problem Statement

To design, model and simulate a traffic light controller for a two-road junction using Verilog HDL, in two forms: (a) a **timer-based** controller whose state changes are governed by a counter, and (b) a **sensor-based** controller whose state changes depend on two sensor inputs. The controllers must start from a known state on reset, must move through the signal sequence in the correct order, and must never show conflicting green signals to both roads.

2.2 Objectives

- To understand the working principle of a digital traffic light controller as a finite state machine.
- To design a six-state timer-based controller with different dwell times for the main phases and the transition phases.
- To design a four-state sensor-based controller whose transitions depend on the inputs SNS1 and SNS2.
- To use parameters for state and colour encoding so that the code stays readable and easy to modify.
- To write testbenches that generate the clock, apply reset and drive the sensor inputs.
- To verify the designs through waveform analysis and to document the observed behaviour.

2.3 Design Requirements

Table 1: Design requirements of the two controllers

Requirement	Timer-based design	Sensor-based design
Clock	Single clock (clk), toggled every 10 time units	Single clock (clk), toggled every 5 time units
Reset	Clear input (clr) returns to S0 and clears the counter	Clear input (clr) returns to S0
Decision logic	Counter compared with SEC5 / SEC1	Sensor inputs SNS1 and SNS2
Outputs	lights[5:0]: three bits per road	TL1[1:0] and TL2[1:0]: two bits per road
Number of states	6 (S0 to S5)	4 (S0 to S3)

3. Theoretical Background

3.1 Sequential Circuits and Finite State Machines

A combinational circuit produces outputs that depend only on the present inputs. A sequential circuit also has memory: its outputs depend on the present inputs and on a stored *state*. A Finite State Machine is the standard model of a sequential circuit. It is defined by a finite set of states, an initial (reset) state, a state-transition function and an output function.

An FSM has three logical blocks: a **state register** built from flip-flops, a **next-state logic** block that decides the state to be loaded on the next clock edge, and an **output logic** block that generates the outputs from the state.

3.2 Moore and Mealy Machines

In a **Moore machine** the outputs depend only on the present state. In a **Mealy machine** the outputs depend on the present state and the present inputs. Both traffic light controllers in this project are Moore machines: the lamp pattern is decided entirely by the current state (the case statement and if-else chain that assign *lights*, TL1 and TL2 use only *state*). Moore outputs change only when the state changes, which gives glitch-free lamp signals.

3.3 Hardware Description Language and Verilog

A Hardware Description Language describes the structure and behaviour of digital hardware. Verilog allows a circuit to be described at the behavioural, RTL, gate and switch levels. At RTL the designer specifies registers and the logic between them. The same code can be simulated to check behaviour and synthesised to hardware.

The important Verilog constructs used in this project are listed below.

Table 2: Verilog constructs used

Construct	Purpose in this project
module / endmodule	Defines the controller as a hardware block with ports.
input wire / output reg	Declares port direction and type. Outputs assigned inside always blocks are declared reg.
reg [n:0]	State register, counter register and outputs.
parameter	Gives names (S0 to S5, red, yellow, green, SEC5, SEC1) to constant values.
always @(posedge clk or posedge clr)	Sequential block: runs on a rising clock edge or a rising clear.
always @(*)	Combinational block: re-evaluated whenever any input changes.
Non-blocking assignment (<=)	Used in sequential blocks so that all registers update together at the clock edge.
Blocking assignment (=)	Used in combinational blocks for next-state and output logic.
case / if-else	Selects next state and output pattern according to the present state.
initial / always #delay	Used in the testbench to generate reset, stimulus and the clock.
`timescale 1ns / 1ps	Sets the simulation time unit (1 ns) and precision (1 ps).

3.4 Counter-Based Timing

A synchronous counter increments once per clock cycle. If a state is to be held for N clock cycles, the controller increments the counter while the count is below a threshold and moves to the next state when the threshold is reached, resetting the counter to zero. This is how the timer-based design generates the long and short signal durations without any external timer.

3.5 Sensor-Driven Control

In a real junction, vehicle detectors (inductive loops, infra-red or camera based) tell the controller whether vehicles are waiting. A sensor-based controller stays in a phase until the relevant sensor indicates that the other road needs service. In the simulation, the sensors are modelled as input signals SNS1 and SNS2 driven from the testbench.

3.6 Synchronous Design and Reset

In synchronous design every state change happens on a clock edge. This makes timing predictable and allows standard timing analysis. Both modules also respond to the *clear* input in the sensitivity list (*posedge clr*), so the controller can be forced to its initial state S0 immediately, even between clock edges. This is called an asynchronous reset and is useful for bringing the hardware to a safe known state after power-up.

4. Tools and Design Elements

4.1 Tools Used

Table 3: Tools and environment

Item	Description
Language	Verilog HDL (IEEE 1364)
Design style	RTL behavioural modelling of Moore state machines
Simulation	HDL simulator with a waveform viewer (time axis in ns)
Time scale	1 ns time unit, 1 ps precision
Documentation	State diagrams, code listings and waveform screenshots from the program assignment

4.2 Signals of the Timer-Based Design

Table 4: Signals of Traffic_Light_Time

Signal	Direction / type	Width	Description
clk	Input wire	1	System clock
clr	Input wire	1	Clear: forces state to S0 and count to 0
lights	Output reg	6	Lamp pattern for both roads
state	Internal reg	3	Present state (S0 to S5)
count	Internal reg	4	Clock-cycle counter used for timing

4.3 Parameters of the Timer-Based Design

Table 5: Parameters of Traffic_Light_Time

Parameter	Value	Meaning
S0	3'b000	Road A red, Road B green (long phase)
S1	3'b001	Road A red, Road B yellow
S2	3'b010	Both roads red (clearance)
S3	3'b011	Road A green, Road B red (long phase)
S4	3'b100	Road A yellow, Road B red
S5	3'b101	Both roads red (clearance)
SEC5	4'b1111 (15)	Threshold for the long phases S0 and S3
SEC1	4'b0011 (3)	Threshold for the short phases S1, S2, S4 and S5

4.4 Signals and Parameters of the Sensor-Based Design

Table 6: Signals of Traffic_Light_Sensor

Signal	Direction / type	Width	Description
clk	Input	1	System clock
clr	Input	1	Clear: forces state to S0
SNS1	Input	1	Sensor input 1, used in state S2
SNS2	Input	1	Sensor input 2, used in state S0
TL1	Output reg	2	Traffic light 1 colour code
TL2	Output reg	2	Traffic light 2 colour code
state, nextstate	Internal reg	2	Present state and next state (S0 to S3)

Table 7: Parameters of Traffic_Light_Sensor

Parameter	Value	Meaning
S0 / S1 / S2 / S3	2'b00 / 2'b01 / 2'b10 / 2'b11	State encoding
green	2'b00	Green lamp code
yellow	2'b01	Yellow lamp code
red	2'b10	Red lamp code

5. Timer-Based Traffic Light Controller

5.1 Design Overview

The module **Traffic_Light_Time** controls a two-road junction using only the clock and the clear input. Its six-bit output *lights[5:0]* is split into two groups of three bits. The upper group *lights[5:3]* is the signal of one road and the lower group *lights[2:0]* is the signal of the other road. Within each group the bits represent red, yellow and green lamps in that order (100 = red, 010 = yellow, 001 = green). This is consistent with the comments in the code, for example S0 = 100001 is "red green" and S3 = 001100 is "green red".

5.2 State Diagram

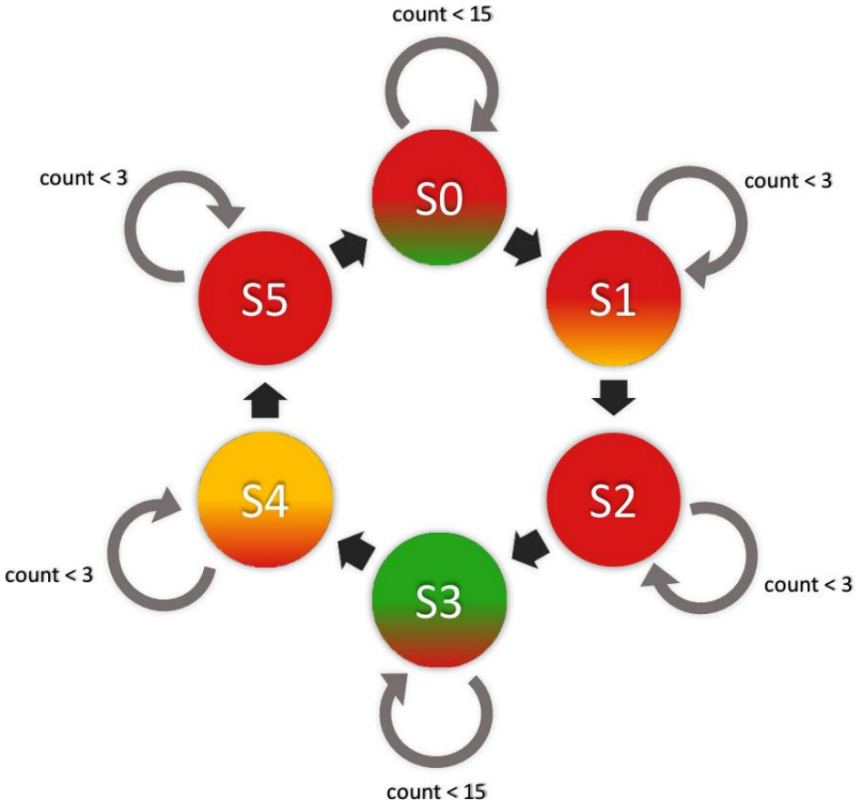


Fig. 1: State diagram of the timer-based controller (from the program assignment)

The diagram shows six states arranged in a ring. Every state has a self-loop labelled *count < 15* (S0 and S3) or *count < 3* (S1, S2, S4 and S5). While the self-loop condition is true the controller remains in the state and the counter increments. When the condition fails the controller takes the arrow to the next state and the counter is cleared. The sequence is therefore S0, S1, S2, S3, S4, S5 and back to S0.

5.3 State Table and Output Encoding

Table 8: State table of Traffic_Light_Time

State	Code	Road A (lights[5:3])	Road B (lights[2:0])	lights[5:0]	Hold threshold	Next state
S0	000	Red (100)	Green (001)	100001	count < 15	S1
S1	001	Red (100)	Yellow (010)	100010	count < 3	S2
S2	010	Red (100)	Red (100)	100100	count < 3	S3
S3	011	Green (001)	Red (100)	001100	count < 15	S4
S4	100	Yellow (010)	Red (100)	010100	count < 3	S5
S5	101	Red (100)	Red (100)	100100	count < 3	S0

The table shows two important safety properties. First, at no time is a green lamp shown to both roads: green appears for Road B only in S0 and for Road A only in S3. Second, between the two green phases there is always a yellow phase followed by an all-red clearance state (S1 then S2, and S4 then S5), which gives vehicles time to clear the junction before the opposing road is released.

5.4 Source Code Explanation

Declarations. The module declares the inputs clk and clr, the six-bit output reg *lights*, a three-bit *state* register and a four-bit *count* register. The states are given names with parameters, and two delay constants are defined: SEC5 = 4'b1111 for the long phases and SEC1 = 4'b0011 for the short phases.

```
1  `timescale 1ns / 1ps
2
3  module Traffic_Light_Time(
4
5    input wire clk,
6      input wire clr,
7      output reg [5:0] lights
8      );
9
10    reg[2:0] state;
11    reg[3:0] count;
12
13    parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010, // states
14              S3 = 3'b011, S4 = 3'b100, S5 = 3'b101;
15
16    parameter SEC5 = 4'b1111, SEC1 = 4'b0011; // delays
17
```

Fig. 2: Timer-based design: module header, registers and parameters

Reset logic. The sequential always block is sensitive to the rising edge of clk or clr. When clr is 1 the state is set to S0 and the count is cleared to 0. Because clr is in the sensitivity list, the reset takes effect immediately and does not wait for a clock edge.

```
18    always @(posedge clk or posedge clr)
19
20       begin
21          if (clr == 1)
22             begin
23                state <= S0;
24                count <= 0;
25             end
26          else
```

Fig. 3: Reset branch of the sequential block

State and count update. When clr is 0, a case statement examines the present state. For S0, if count is less than SEC5 the machine remains in S0 and increments the counter; otherwise it moves to S1 and clears the counter. The remaining states follow the same pattern with their own thresholds and next states (SEC5 for S3, SEC1 for S1, S2, S4 and S5).

```
27             case(state)
28                S0: if(count < SEC5)
29                         begin
30                            state <= S0;
31                            count <= count + 1;
32                         end
33                      else
34                         begin
35                            state <= S1;
36                            count <= 0;
37                         end
```

Fig. 4: State and count update logic for S0 (remaining states follow the same pattern)

Output logic. A separate combinational block, always @(*), decodes the present state into the six-bit lamp pattern. A default branch assigns the S0 pattern so that the outputs are always defined, even if the state register ever holds an unused code (110 or 111).

```
94    always @(*)
95       begin
96
97          case(state)
98
99             S0: lights = 6'b100001; // red green
100            S1: lights = 6'b100010; // red yellow
101            S2: lights = 6'b100100; // red red
102            S3: lights = 6'b001100; // green red
103            S4: lights = 6'b010100; // yellow red
104            S5: lights = 6'b100100; // red red
105            default lights = 6'b100001; // red green
106
107         endcase
108      end
109
110   endmodule
```

Fig. 5: Output decoding of lights[5:0] from the present state

5.5 Timing Analysis

In the code the counter is incremented while *count* < *threshold*, so a state is held for the values 0, 1, ..., threshold, which is threshold + 1 clock cycles. For the long phases this is 15 + 1 = 16 cycles and for the short phases 3 + 1 = 4 cycles. The testbench toggles the clock every 10 ns, so the clock

period is 20 ns.

Table 9: Dwell time of each state

State	Threshold	Clock cycles in state	Duration at 20 ns period
S0	15	16	320 ns
S1	3	4	80 ns
S2	3	4	80 ns
S3	15	16	320 ns
S4	3	4	80 ns
S5	3	4	80 ns
Full cycle		**48**	**960 ns**

The names SEC5 and SEC1 indicate a "five-unit" long delay and a "one-unit" short delay. In a real installation the clock would be derived from a slower timing source, or a clock divider would be added so that one count equals a convenient real-time interval such as a second; the state machine itself would remain unchanged.

5.6 Working Steps of the Timer-Based Controller

- Assert clr: state becomes S0, count becomes 0 and the lights show Road A red, Road B green.
- On each rising clock edge, the controller compares count with the threshold of the present state.
- If count is below the threshold, the state is held and count is incremented.
- When count reaches the threshold, the state moves to the next state and count is cleared.
- The output block continuously decodes the present state into lights[5:0].
- After S5 the controller returns to S0 and the cycle repeats indefinitely.

6. Sensor-Based Traffic Light Controller

6.1 Design Overview

The module **Traffic_Light_Sensor** adds input-dependent behaviour. It has four states (S0 to S3) and two sensor inputs, SNS1 and SNS2. Instead of a counter, the controller uses a *nextstate* register that is computed by combinational logic from the present state and the sensors. Two 2-bit outputs, TL1 and TL2, give the colour of each light using the codes green = 00, yellow = 01 and red = 10.

6.2 State Diagram

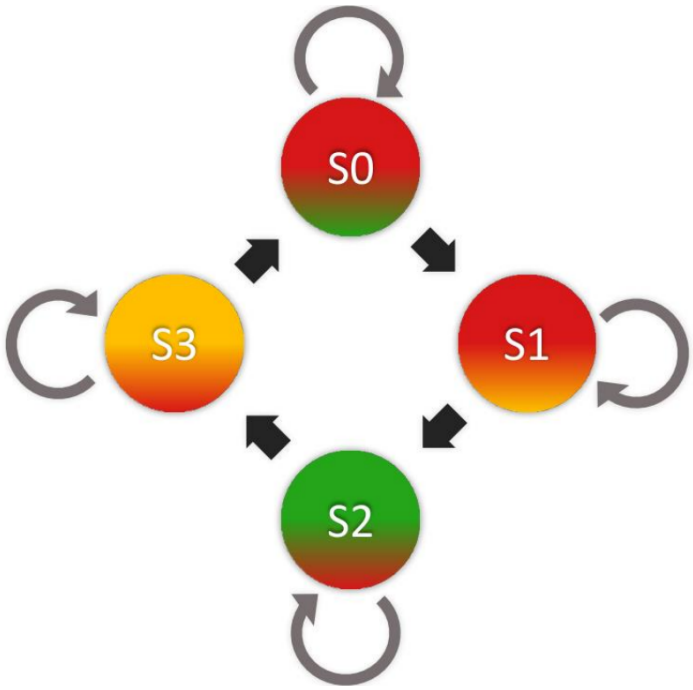


Fig. 6: State diagram of the sensor-based controller (from the program assignment)

The four states form a ring (S0, S1, S2, S3). S0 and S2 are the two green phases and can hold themselves (self-loops) while the sensor condition is not satisfied. S1 and S3 are the yellow phases and always move on at the next clock edge.

6.3 State Transition Table

Table 10: State transition table of Traffic_Light_Sensor

Present state	Condition	Next state	Remark
S0	SNS2 = 1	S1	Request from the other road detected: begin changeover
S0	SNS2 = 0	S0	Hold the current green
S1	(none)	S2	Yellow lasts one clock cycle
S2	SNS1 = 1	S3	Request detected: begin changeover
S2	SNS1 = 0	S2	Hold the current green
S3	(none)	S0	Yellow lasts one clock cycle
default	(none)	S0	Safe recovery for unused codes

6.4 Output Table

Table 11: Output encoding of TL1 and TL2

State	TL1	TL1 code	TL2	TL2 code
S0	Green	00	Red	10
S1	Yellow	01	Red	10
S2	Red	10	Green	00
S3 (else branch)	Red	10	Yellow	01

As in the timer-based design, the two roads are never green together, and each green phase is followed by a yellow phase before the other road is released. The outputs depend only on the present state, so this is also a Moore machine.

6.5 Source Code Explanation

Declarations and state register. The module header lists clk, clr, SNS1 and SNS2 as inputs and TL1 and TL2 as two-bit output registers. Parameters define the four states and the three colours. The sequential block, sensitive to posedge clk and posedge clr, loads S0 on clear and otherwise loads *nextstate*.

```
1  `timescale 1ns / 1ps
2
3  module Traffic_Light_Sensor(
4      input clk,
5      input clr,
6      input SNS1,
7      input SNS2,
8      output reg[1:0] TL1, TL2
9      );
10
11 reg [1:0] state, nextstate;
12
13 parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
14 parameter green = 2'b00, yellow = 2'b01, red = 2'b10;
15
```

```
17 always@(posedge clk, posedge clr)
18    begin
19       if(clr) state <= S0;
20       else state <=nextstate;
21    end
```

Fig. 7: Sensor-based design: ports, registers, parameters and state register

Next-state logic. A combinational block, always @(*), implements the transition table. In S0 it checks SNS2, in S2 it checks SNS1, while S1 and S3 move unconditionally. The default branch returns to S0.

```
24 always@(*)
25    begin
26       case(state)
27          S0: if(SNS2) nextstate = S1;
28              else nextstate = S0;
29          S1: nextstate = S2;
30          S2: if(SNS1) nextstate = S3;
31              else nextstate = S2;
32          S3: nextstate = S0;
33          default: nextstate = S0;
34       endcase
35    end
```

Fig. 8: Next-state (transition) logic

Output logic. A second combinational block assigns TL1 and TL2 from the present state using an if-else chain. The final else covers S3 (and, as a safe fallback, any unused code): TL1 red and TL2 yellow.

```
38  always@(*)
39     begin
40        if(state == S0)
41           begin
42              TL1 = green;
43              TL2 = red;
44           end
45        else if(state == S1)
46           begin
47              TL1 = yellow;
48              TL2 = red;
49           end
50        else if(state == S2)
51           begin
52              TL1 = red;
53              TL2 = green;
54           end
55        else begin
56              TL1 = red;
57              TL2 = yellow;
58              end
59     end
60  endmodule
```

Fig. 9: Output logic for TL1 and TL2

6.6 Working Steps of the Sensor-Based Controller

- Assert clr: the state becomes S0, so TL1 is green and TL2 is red.
- While SNS2 is 0 the controller holds S0. When SNS2 becomes 1 the next clock edge moves it to S1 (TL1 yellow).
- The next clock edge moves it from S1 to S2: TL1 red and TL2 green.
- While SNS1 is 0 the controller holds S2. When SNS1 becomes 1 the next clock edge moves it to S3 (TL2 yellow).
- The next clock edge moves it from S3 back to S0 and the cycle can repeat.

7. Testbench and Simulation Results

7.1 Timer-Based Testbench

The testbench for the timer-based design initialises clk and clr to 0 and then waits for 100 time units. A separate always block toggles the clock every 10 time units (*#10 clk = ~clk*), giving a 20 ns clock period. The simulation is then run long enough in the simulator to display more than one full signal sequence.

```
15    initial begin
16
17        clk = 0;
18        clr = 0;
19
20        #100;
21
22    end
23
24    always
25
26    #10 clk = ~clk;
27
28  endmodule
```

Fig. 10: Timer-based testbench (clock generation and initialisation)

7.2 Timer-Based Simulation Waveform

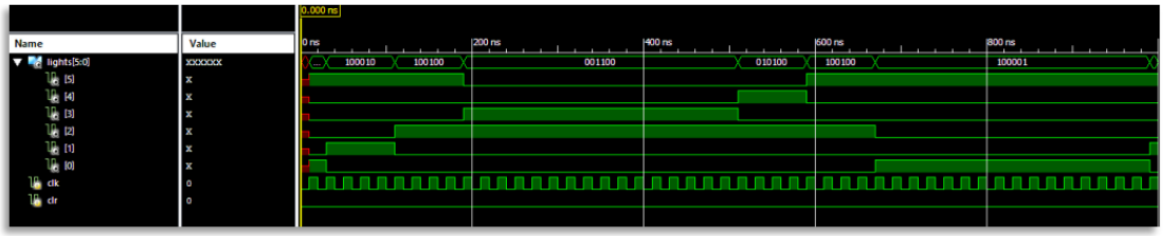


Fig. 11: Simulation waveform of lights[5:0], with each bit, clk and clr

The waveform shows the six-bit bus *lights* as it moves through the following patterns in order: 100010, 100100, 001100, 010100, 100100 and 100001. These correspond exactly to states S1, S2, S3, S4, S5 and S0 in the state table, confirming the ring sequence. The wide 001100 segment (green for one road) lasts much longer than the other segments, matching the 16-cycle long phase against the 4-cycle short phases. The individual bits lights[5] to lights[0] in the lower part of the waveform show which lamp of each road is on at every moment.

Table 12: Verification of the timer-based design

Check	Expected behaviour	Observed in waveform
Order of states	S0, S1, S2, S3, S4, S5, S0 ...	Patterns appear in the ring order
Long phase	Road with green held for 16 cycles (320 ns)	001100 segment is the widest segment
Short phases	Yellow and all-red phases last 4 cycles (80 ns)	Narrow segments for 100010, 100100, 010100
No conflict	Both roads never green together	Green bits lights[0] and lights[3] are never high together
Clock	Period 20 ns	clk toggles every 10 ns

7.3 Sensor-Based Testbench

The sensor testbench applies a clear pulse at the start and then drives the sensors with a defined pattern. The clock is generated by *forever #(10/2) clk = ~clk*, so the period is 10 ns and rising edges occur at 0, 10, 20, ... ns.

```
26    initial
27       begin
28
29          clr = 1;
30          #13
31          clr = 0;
32
33
34          SNS1 = 0; SNS2 = 0; #(10)
35          SNS1 = 0; SNS2 = 1; #(40)
36          SNS1 = 0; SNS2 = 0; #(40)
37          SNS1 = 1; SNS2 = 0; #(30)
38          SNS1 = 0; SNS2 = 0; #(20)
39          SNS1 = 0; SNS2 = 1;
40
41       end
42
43    always
44       begin
45          clk = 1;
46          forever #(10/2) clk = ~clk;
47       end
48
49
50 endmodule
```

Fig. 12: Sensor-based testbench: reset and sensor stimulus

The stimulus can be summarised in a table with the absolute simulation time of every change.

Table 13: Sensor testbench stimulus timeline

Time (ns)	clr	SNS1	SNS2	Description
0 to 13	1	x	x	Reset applied, state forced to S0
13 to 23	0	0	0	Reset released, no sensor active
23 to 63	0	0	1	SNS2 active for 40 ns
63 to 103	0	0	0	No sensor active for 40 ns
103 to 133	0	1	0	SNS1 active for 30 ns
133 to 153	0	0	0	No sensor active for 20 ns
153 onwards	0	0	1	SNS2 active again

7.4 Sensor-Based Simulation Waveform

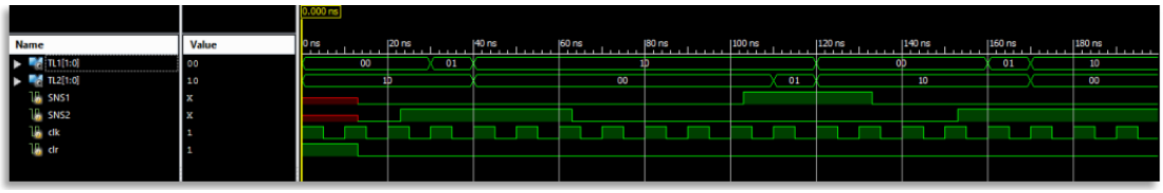


Fig. 13: Simulation waveform of TL1, TL2, SNS1, SNS2, clk and clr

Using the stimulus timeline, the clock edges and the transition table, the expected behaviour of the controller can be traced edge by edge. The state is read just after each rising edge.

Table 14: Edge-by-edge trace of the sensor-based controller

Clock edge (ns)	Sensors (SNS1, SNS2)	State after edge	TL1	TL2	Comment
0 to 10	x, x	S0	Green (00)	Red (10)	Held in reset (clr = 1)
20	0, 0	S0	Green (00)	Red (10)	SNS2 = 0: hold S0
30	0, 1	S1	Yellow (01)	Red (10)	SNS2 = 1 seen: S0 to S1
40	0, 1	S2	Red (10)	Green (00)	S1 always goes to S2
50 to 100	0, 0	S2	Red (10)	Green (00)	SNS1 = 0: hold S2
110	1, 0	S3	Red (10)	Yellow (01)	SNS1 = 1 seen: S2 to S3
120	1, 0	S0	Green (00)	Red (10)	S3 always goes to S0
130 to 150	1/0, 0	S0	Green (00)	Red (10)	SNS2 = 0: hold S0
160	0, 1	S1	Yellow (01)	Red (10)	SNS2 = 1 seen again: S0 to S1
170	0, 1	S2	Red (10)	Green (00)	S1 always goes to S2

The waveform shows TL1 and TL2 changing between the codes 00, 01 and 10 only at times that line up with the clock edges following changes of SNS1 and SNS2, which is the behaviour predicted by the trace. In particular, the controller stays in S0 and S2 as long as the corresponding sensor is inactive, and the yellow phases S1 and S3 last for exactly one clock cycle.

7.5 Summary of Results

- Both controllers start in S0 after reset and follow the intended light sequence.
- The timer-based controller shows a fixed repeating 48-cycle sequence with long (16-cycle) and short (4-cycle) phases.
- The sensor-based controller holds its green phases until the relevant sensor becomes active and then passes through a one-cycle yellow phase.
- No state of either controller gives green to both roads at the same time.
- All outputs are decoded from the present state, so the lamp signals change only on clock edges.

8. Comparison of the Two Designs

The two controllers solve the same problem with different control strategies. The timer-based design is predictable and simple to analyse; the sensor-based design is more adaptive because it reacts to the traffic condition.

Table 15: Comparison of the timer-based and sensor-based controllers

Parameter	Timer-based (Traffic_Light_Time)	Sensor-based (Traffic_Light_Sensor)
Control principle	Fixed time slots measured by a counter	Transitions decided by sensor inputs
Inputs	clk, clr	clk, clr, SNS1, SNS2
Outputs	lights[5:0]	TL1[1:0], TL2[1:0]
Number of states	6	4
State register width	3 bits	2 bits
Extra registers	4-bit counter	nextstate register (2 bits)
Dwell time	Fixed: 16 or 4 cycles	Variable in S0 and S2, one cycle in S1 and S3
All-red clearance state	Yes (S2 and S5)	No separate all-red state
Machine type	Moore	Moore
Response to traffic	None: same sequence always	Adapts to sensor inputs
Best suited to	Junctions with steady, predictable traffic	Junctions with variable or low traffic
Design complexity	Simple, but needs counter logic	Simple state logic, needs sensor interface

A practical controller can combine both ideas: a counter can enforce a minimum green time for safety, while sensors decide whether to extend the green or to change over. This combination is discussed as future scope in Chapter 9.

9. Advantages, Limitations, Applications and Future Scope

9.1 Advantages

- The signal sequence is described clearly by a small number of states, making the design easy to understand and debug.
- Parameters give meaningful names to states and lamp codes, so changes such as a different delay need editing of one line only.
- The timer-based design gives fully predictable cycle timing and an all-red clearance interval.
- The sensor-based design can respond to traffic demand and avoids holding a green phase longer than necessary.
- Moore-style outputs are decoded only from the state, so the lamp outputs are free from input glitches.
- Asynchronous clear brings the controller to a known state immediately.
- Simulation allows the logic to be verified completely before any hardware is built.

9.2 Limitations

- The timer-based design cannot adapt to actual traffic, so a road with no vehicles still receives its full green time.
- The sensor-based design holds a green phase indefinitely while its sensor stays inactive, and has no minimum or maximum green time.
- The sensor-based design has no separate all-red clearance state between phases.
- Only two roads are modelled; pedestrian crossings, turning lanes and emergency-vehicle priority are not included.
- The designs are verified by simulation only and have not been implemented on an FPGA or tested at a real junction.
- The sensor inputs are assumed to be clean, synchronised digital signals; real sensors would need debouncing and synchronisation.

9.3 Applications

- Teaching digital design, finite state machines and Verilog HDL.
- Basic traffic signal controllers for road intersections, after suitable safety validation.
- Railway-crossing, parking-lot and one-lane-road control, which use the same alternating-right-of-way principle.
- Industrial sequencing controllers where a fixed sequence of timed phases must be repeated.
- A starting template for any Moore FSM with timer and input-triggered transitions.

This project is an educational model. It is not, by itself, a certified road traffic control system, which would require extensive safety analysis, fail-safe hardware and regulatory approval.

9.4 Future Scope

- Implement the designs on an FPGA board with LEDs for the lamps and push buttons or IR sensors for SNS1 and SNS2.
- Add a clock divider so that one count corresponds to a real second, and make the green times programmable.
- Combine the two controllers: minimum green time from a counter plus sensor-based extension.
- Add an all-red clearance state and a maximum green time to the sensor-based design.
- Extend the design to a four-way junction with pedestrian request buttons and an emergency-vehicle override.
- Add a fail-safe flashing-red or flashing-yellow mode for fault conditions.
- Write a self-checking testbench with assertions, and synthesise the design to report area and timing.

10. Conclusion

This project presented the design and simulation of a traffic light controller in Verilog HDL using two control strategies. The **timer-based controller** uses a six-state finite state machine and a 4-bit counter with two thresholds (15 and 3) to hold the long phases for 16 clock cycles and the short phases for 4 clock cycles, repeating a 48-cycle sequence. The **sensor-based controller** uses a four-state machine in which the sensors SNS2 and SNS1 decide when the green phases of S0 and S2 end, with one-cycle yellow phases in between.

Through the state diagrams, state tables, source-code explanation, testbench stimulus and waveform analysis, the report shows how states, parameters, counters, combinational next-state logic and output decoding combine to describe real control hardware. Both designs follow the Moore model, never display green to both roads at the same time, and start from a known state on reset.

The work reinforced several important RTL design practices: separating the sequential state register from the combinational next-state and output logic, using parameters for readability, providing a default branch for unused states, and verifying behaviour with a testbench and waveform viewer. The suggested future enhancements, such as FPGA implementation, programmable timing, combined timer-plus-sensor control and pedestrian support, offer a clear path to extend this learning project towards a more complete controller.

References

[1] Unlox Academy, *Project 01: Traffic Light Controller (Timer and Sensor),* VLSI program assignment (Minor_Project_02_September_VLSI), September 2026.

[2] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.

[3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall.

[4] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson.

Appendix A: Complete Timer-Based Source Code

The listing below gives the complete module *Traffic_Light_Time*. The header, S0 branch, reset logic and output block are as shown in the assignment screenshots in Chapter 5; the case branches for S1 to S5 follow the same pattern as S0 with the thresholds and next states given in the state table (Table 8).

```
`timescale 1ns / 1ps

module Traffic_Light_Time(
    input wire clk,
    input wire clr,
    output reg [5:0] lights
);

    reg [2:0] state;
    reg [3:0] count;

    parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010,   // states
              S3 = 3'b011, S4 = 3'b100, S5 = 3'b101;

    parameter SEC5 = 4'b1111, SEC1 = 4'b0011;          // delays

    // ---- state and counter update ----
    always @(posedge clk or posedge clr)
    begin
        if (clr == 1)
        begin
            state <= S0;
            count <= 0;
        end
        else
            case (state)
                S0: if (count < SEC5) begin state <= S0; count <= count + 1; end
                    else              begin state <= S1; count <= 0;         end
                S1: if (count < SEC1) begin state <= S1; count <= count + 1; end
                    else              begin state <= S2; count <= 0;         end
                S2: if (count < SEC1) begin state <= S2; count <= count + 1; end
                    else              begin state <= S3; count <= 0;         end
                S3: if (count < SEC5) begin state <= S3; count <= count + 1; end
                    else              begin state <= S4; count <= 0;         end
                S4: if (count < SEC1) begin state <= S4; count <= count + 1; end
                    else              begin state <= S5; count <= 0;         end
                S5: if (count < SEC1) begin state <= S5; count <= count + 1; end
                    else              begin state <= S0; count <= 0;         end
                default: state <= S0;
            endcase
    end

    // ---- output decoding ----
    always @(*)
    begin
        case (state)
            S0: lights = 6'b100001; // red green
            S1: lights = 6'b100010; // red yellow
            S2: lights = 6'b100100; // red red
            S3: lights = 6'b001100; // green red
            S4: lights = 6'b010100; // yellow red
            S5: lights = 6'b100100; // red red
            default lights = 6'b100001; // red green
        endcase
    end

endmodule
```

Timer-based testbench stimulus (as shown in the assignment):

```
initial begin
    clk = 0;
    clr = 0;
    #100;
end

always
    #10 clk = ~clk;

endmodule
```

Appendix B: Complete Sensor-Based Source Code

The listing below is the module *Traffic_Light_Sensor* with its testbench stimulus, as shown in the assignment.

```
`timescale 1ns / 1ps

module Traffic_Light_Sensor(
    input clk,
    input clr,
    input SNS1,
    input SNS2,
    output reg [1:0] TL1, TL2
);

reg [1:0] state, nextstate;

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
parameter green = 2'b00, yellow = 2'b01, red = 2'b10;

// ---- state register ----
always @(posedge clk, posedge clr)
begin
    if (clr) state <= S0;
    else     state <= nextstate;
end

// ---- next-state logic ----
always @(*)
begin
    case (state)
        S0: if (SNS2) nextstate = S1;
            else      nextstate = S0;
        S1: nextstate = S2;
        S2: if (SNS1) nextstate = S3;
            else      nextstate = S2;
        S3: nextstate = S0;
        default: nextstate = S0;
    endcase
end

// ---- output logic ----
always @(*)
begin
    if (state == S0)
    begin
        TL1 = green;  TL2 = red;
    end
    else if (state == S1)
    begin
        TL1 = yellow; TL2 = red;
    end
    else if (state == S2)
    begin
        TL1 = red;    TL2 = green;
    end
    else begin
        TL1 = red;    TL2 = yellow;
    end
end

endmodule
```

Sensor-based testbench stimulus (as shown in the assignment):

```
initial
```

```
begin
    clr = 1;
    #13
    clr = 0;

    SNS1 = 0; SNS2 = 0; #(10)
    SNS1 = 0; SNS2 = 1; #(40)
    SNS1 = 0; SNS2 = 0; #(40)
    SNS1 = 1; SNS2 = 0; #(30)
    SNS1 = 0; SNS2 = 0; #(20)
    SNS1 = 0; SNS2 = 1;
end

always
begin
    clk = 1;
    forever #(10/2) clk = ~clk;
end

endmodule
```

Appendix C: Glossary

Term	Meaning
FSM	Finite State Machine: a model with a finite number of states and rules for moving between them.
Moore machine	FSM whose outputs depend only on the present state.
RTL	Register Transfer Level: description of hardware as registers and the logic between them.
HDL	Hardware Description Language such as Verilog or VHDL.
Testbench	Simulation-only module that supplies clock, reset and inputs to the design under test.
Sensitivity list	List of signals in an always block whose changes trigger the block.
Non-blocking assignment	Assignment (<=) that updates registers together at the end of the time step; used for sequential logic.
Asynchronous reset	Reset that acts immediately when asserted, independent of the clock.
Time scale	Directive defining the unit and precision of simulation time.